

Comp 3005: Final Project Report

Fitness Club Application

Group Members:

Naomi Amaya Lovett (101223562)

Natalia Gomez (101256528)

Charita Ponnada (101303429)

Professor:

Dr. Abdelghny Orogat

Submission Date:

1/12/2025

Youtube Submission Link:

https://www.youtube.com/watch?v=n_mrF7VLnLE

Github Submission Link:

<https://github.com/naomiamayaa/COMP3005-Final-Project>

Intro:	3
ER Model	3
Relational mapping:	3
Normalization:	4
ORM usage:	5
Conclusion:	5
Diagrams:	6

Intro:

This project was completed by a group of three students, and together we implemented the twelve required functions across the Member, Trainer, and Admin roles. We focused on all the core functional requirements of the fitness club system such as class scheduling, user management, availability handling, room booking, health tracking, and maintenance while leaving out only the optional finance section. All twelve features were fully coded and tested using SQLAlchemy ORM. Because we split the work, each member contributed to database design, functionality, and CLI development. All features were implemented through a role-based CLI, with separate menus for Members, Trainers, and Admins.

ER Model: **Please refer to diagrams at the end of the document or in the Documents folder of the github repository.

For this project, we created an ER model that represents all the main parts of a fitness club. The main entities are Users, Members, Trainers, Admins, Rooms, Classes, Health Metrics, Member Goals, Equipment, and Maintenance Records.

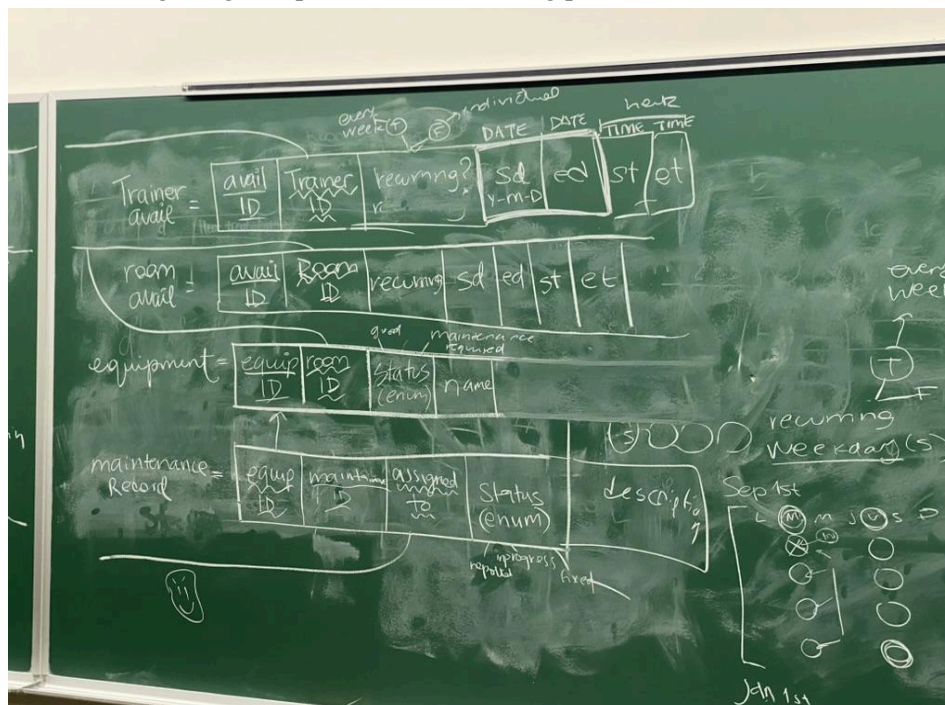
Users is the parent entity, and it specializes into Member, Trainer, and Admin. For example, a Trainer is still a User but has extra features like availability blocks. Members also connect to other entities such as MemberGoals and HealthMetrics, since a member can have many goals and many measurements over time.

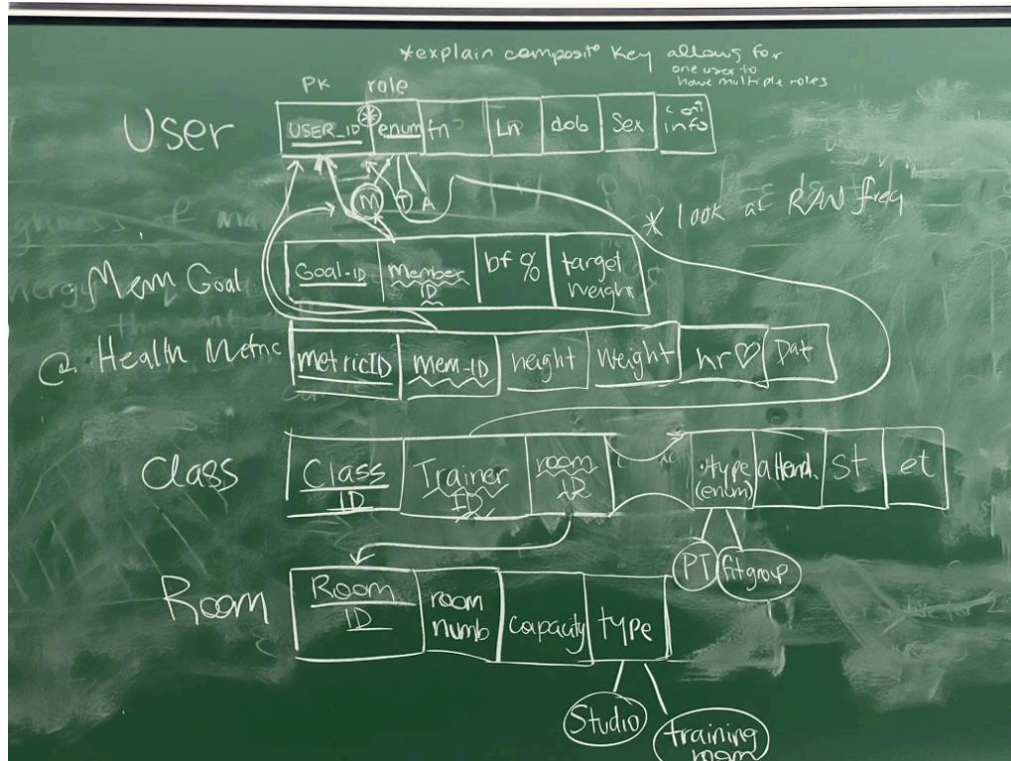
Separate tables are used for availability (TrainerAvailability and RoomAvailability) so that each person or room can have multiple time blocks. This makes it easy to enforce no overlapping times.

Relational mapping:

After building the ER diagram, we converted each entity into a relational table. Every table has a primary key called id.

The following images depict our brainstorming process:





Normalization:

All tables in the Fitness Club database satisfy First, Second, and Third Normal Form. The schema is in 1NF because every column stores a single, atomic value, and there are no repeating groups or multi-valued attributes. Each table also has a primary key called id, which ensures every row is uniquely identified. The design automatically satisfies 2NF because none of the tables use a composite primary key; since every table has only one primary key column, it is impossible for a non-key attribute to depend on just part of a key. Every non-key attribute depends on the entire primary key by definition.

The schema also satisfies 3NF because there are no transitive dependencies in any table. For example, in the USERS table, attributes such as email, first_name, last_name, role, and sex all depend only on id, and none of these attributes determine one another. Similar reasoning applies to the other tables.

MEMBER_GOALS stores only goal information and a foreign key to the member, but no member attributes are repeated, so there is no chain of dependencies. HEALTH_METRICS stores measurements like weight and bpm, all of which depend solely on the measurement entry, not on each other. In CLASSES, foreign keys like trainer_id and room_id do not determine any of the other fields within the same table. CLASS_REGISTRATIONS does not contain any attributes that determine other non-key attributes. ROOMS, TRAINER_AVAILABILITY, ROOM_AVAILABILITY, EQUIPMENT, and MAINTENANCE_RECORDS also contain only attributes that depend directly on their primary key. Because no table includes an attribute that depends on another non-key attribute, the schema contains no transitive dependencies.

Based on these observations, the entire Fitness Club schema is normalized to both 2NF and 3NF.

In SQLAlchemy ORM, each table is represented by a Python class. For example, the Users table is mapped like this:

```
class Users(Base):
    __tablename__ = "users"
    id = Column(Integer, primary_key=True)
    email = Column(String, unique=True)
    role = Column(Enum(UserRole))
```

ORM usage:

In this project we used SQLAlchemy ORM to handle almost all of the communication between our Python code and the PostgreSQL database. Instead of writing raw SQL queries, we represented every table as a Python class, such as Users, Classes, Rooms, Equipment, and so on. Each attribute in these classes maps to a database column, and the relationships (like one-to-many or many-to-many) were created using SQLAlchemy's relationship() and ForeignKey() features.

For example, creating a new user didn't require an INSERT statement; we just wrote:

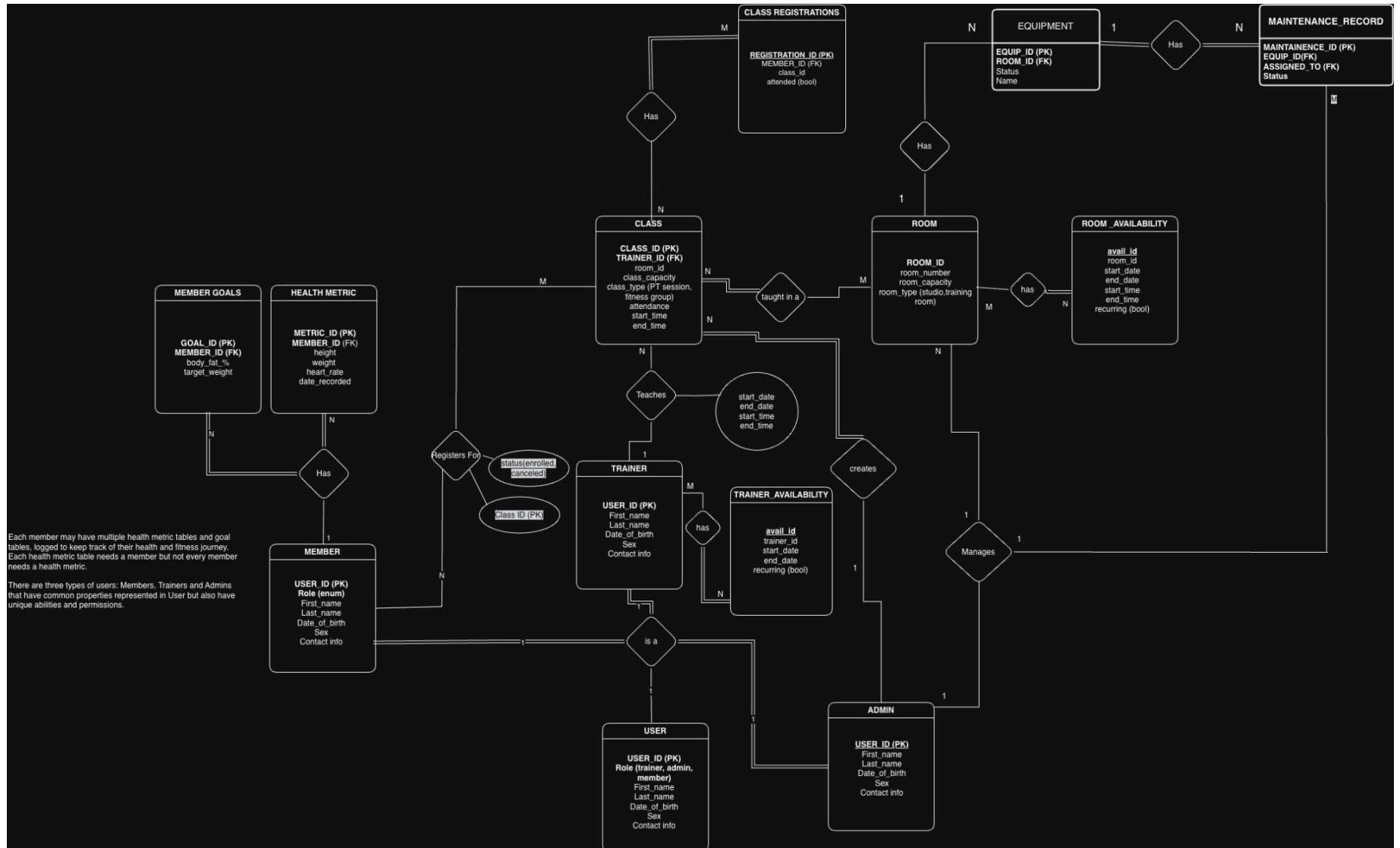
```
new_user = Users(email="test@example.com", role=UserRole.MEMBER, ...)
session.add(new_user)
session.commit()
```

and SQLAlchemy automatically generated the SQL behind the scenes. We also used ORM queries such as session.query(Classes).filter(...) to list upcoming sessions, find available trainers or rooms, or check for booking conflicts. The ORM also helped enforce constraints defined in the model classes, like unique emails, enum types for roles, and foreign-key relationships. Using SQLAlchemy allowed us to focus more on implementing the required functionality (like scheduling, maintenance, and goal tracking) without worrying about writing or debugging SQL manually.

Conclusion:

In this project, our group designed and implemented a complete Fitness Club Management System using PostgreSQL and SQLAlchemy ORM. We created a fully normalized database schema, built an ER model that accurately represents real-world relationships, and mapped it into relational tables using ORM classes. Across the Member, Trainer, and Admin roles, we implemented twelve working features that cover scheduling, availability, registrations, health tracking, equipment management, and maintenance logging. Although we did not include the optional finance component, we successfully met all required functional and technical specifications. Overall, this project helped us apply database design principles, normalization, and practical software development skills to create a system that works reliably and reflects real operational needs in a fitness club environment.

Diagrams:



Comp 3005 Final Project Database Schema

